

Proyecto 2 — LinProd

Guía de Implementación

Módulo 4: Reportería PDF

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE-5507 — Modelación Hardware Software Orientado a Objetos
I Semestre 2026

Archivo a implementar: `linprod/reports.py`

Contexto y rol del Módulo 4

LinProd está dividido en cuatro módulos con una cadena de dependencia estricta en un solo sentido:

```
model.py ← simulator.py ← app.py  
reports.py ← tú aquí
```

Tu trabajo consiste únicamente en implementar `linprod/reports.py`. No debes modificar ningún otro archivo. El módulo recibe los datos del simulador mediante la función `Simulator.get_stats()` y genera un reporte en PDF usando la librería `reportlab`.

Regla de oro

`reports.py` solo importa de `linprod.simulator` (y por transitividad de `linprod.model`). No importa nada de `app.py` ni de `tkinter`.

Instalación de reportlab

`reportlab` es la única dependencia no-stdlib del proyecto. Para instalarla ejecuta en la terminal:

```
pip install reportlab  
# o si el sistema usa pip3:  
pip3 install reportlab
```

Para verificar que quedó instalada correctamente:

```
python3 -c "import reportlab; print(reportlab.Version)"  
# Salida esperada: algo como 4.x.x
```

Datos de entrada: `Simulator.get_stats()`

La función `get_stats()` del simulador es la **única fuente de datos** para el reporte. Retorna un diccionario con la siguiente estructura:

Claves del diccionario principal

Clave	Tipo	Descripción
<code>total_cycles_run</code>	<code>int</code>	Ciclos totales ejecutados hasta terminar
<code>products_total</code>	<code>int</code>	Total de productos que entraron a la línea
<code>products_completed</code>	<code>int</code>	Productos que completaron toda la línea
<code>first_completion_cycle</code>	<code>int None</code>	Ciclo en que salió el primer producto
<code>last_completion_cycle</code>	<code>int None</code>	Ciclo en que salió el último producto
<code>avg_completion_time</code>	<code>float</code>	Tiempo promedio de paso (ciclos)
<code>total_processing_time</code>	<code>int</code>	Suma de tiempos de todos los productos
<code>bottleneck_task</code>	<code>str None</code>	Nombre de la tarea cuello de botella
<code>bottleneck_process</code>	<code>str None</code>	Proceso al que pertenece el cuello de botella
<code>avg_wait_to_start</code>	<code>float</code>	Espera promedio global antes de iniciar tarea
<code>worst_wait_task</code>	<code>str None</code>	Tarea con mayor espera promedio
<code>worst_wait_process</code>	<code>str None</code>	Proceso de la tarea con mayor espera
<code>worst_wait_value</code>	<code>float</code>	Valor de esa mayor espera (ciclos)
<code>task_stats</code>	<code>list[dict]</code>	Estadísticas por tarea (ver §3.2)
<code>product_stats</code>	<code>list[dict]</code>	Estadísticas por producto (ver §3.3)

Estructura de cada elemento en `task_stats`

Cada elemento de la lista `task_stats` es un diccionario con:

Clave	Tipo	Descripción
<code>task_name</code>	<code>str</code>	Nombre de la tarea
<code>process_name</code>	<code>str</code>	Nombre del proceso al que pertenece
<code>process_time</code>	<code>int</code>	Tiempo de procesamiento configurado (ciclos)
<code>products_processed</code>	<code>int</code>	Cuántos productos procesó esta tarea
<code>avg_wait_cycles</code>	<code>float</code>	Espera promedio de productos en la cola
<code>busy</code>	<code>bool</code>	Si la tarea estaba ocupada al terminar

Estructura de cada elemento en `product_stats`

Clave	Tipo	Descripción
<code>product_id</code>	<code>int</code>	Identificador del producto (1, 2, 3...)
<code>entry_time</code>	<code>int</code>	Ciclo en que entró a la línea
<code>exit_time</code>	<code>int None</code>	Ciclo en que salió (<code>None</code> si no terminó)
<code>is_completed</code>	<code>bool</code>	Si completó toda la línea
<code>total_time</code>	<code>int None</code>	Ciclos totales de entrada a salida
<code>task_history</code>	<code>list[dict]</code>	Historial de paso por cada tarea

El historial de tareas (`task_history`) de cada producto es una lista de diccionarios, uno por cada tarea que procesó. Cada entrada tiene: `task_name`, `process_name`, `start_cycle`, `end_cycle`, `wait_cycles`.

Contenido mínimo del reporte (según el enunciado)

El enunciado del proyecto exige que el reporte presente los siguientes datos:

- ✓ Tiempo en que el **primer producto** logró terminar con éxito toda la línea (`first_completion_cycle`).
- ✓ Tiempo en que el **último producto** logró terminar con éxito toda la línea (`last_completion_cycle`).
- ✓ **Tiempo promedio** de paso por toda la línea (`avg_completion_time`).
- ✓ Proceso que generó el mayor **congestionamiento** (**cuello de botella**: `bottleneck_process / bottleneck_task`).
- ✓ **Tiempo promedio de espera** de productos para iniciar una tarea (`avg_wait_to_start`).
- ✓ Proceso y tarea con **mayor tiempo de espera** (`worst_wait_process / worst_wait_task`).
- ✓ **Tiempo total** del procesamiento de todos los productos (`total_processing_time`).
- ✓ Cualquier otra estadística que el grupo considere necesaria.

Se recomienda adicionalmente incluir la **tabla de detalle por tarea** (`task_stats`) y la **tabla de detalle por producto** (`product_stats`).

Estructura recomendada de `reports.py`

```

1  """
2  reports.py - Módulo 4: Generación de reportes PDF
3  Proyecto LinProd | CE-5507 | I Semestre 2026
4  """
5
6  from __future__ import annotations
7  from typing import Optional

```

```

8
9 from reportlab.lib.pagesizes import letter
10 from reportlab.lib.styles import getSampleStyleSheet,
    ParagraphStyle
11 from reportlab.lib.units import cm
12 from reportlab.lib import colors
13 from reportlab.platypus import (
14     SimpleDocTemplate, Paragraph, Spacer,
15     Table, TableStyle, HRFlowable,
16 )
17 from reportlab.lib.enums import TA_CENTER, TA_LEFT
18
19
20 def generate_report(stats: dict, output_path: str) -> None:
21     """
22     Genera el reporte PDF de la simulación.
23
24     Parámetros
25     -----
26     stats : dict
27         Diccionario devuelto por Simulator.get_stats().
28     output_path : str
29         Ruta completa del archivo PDF a generar.
30         Ejemplo: "/home/usuario/reporte_linprod.pdf"
31     """
32     doc = SimpleDocTemplate(
33         output_path,
34         pagesize=letter,
35         leftMargin=2.5 * cm,
36         rightMargin=2.5 * cm,
37         topMargin=2.5 * cm,
38         bottomMargin=2.5 * cm,
39     )
40
41     story = [] # lista de elementos (flowables)
42     styles = _build_styles()
43
44     # 1. Portada / encabezado
45     _add_header(story, styles, stats)
46
47     # 2. Resumen general
48     _add_summary(story, styles, stats)
49
50     # 3. Cuello de botella y esperas
51     _add_bottleneck(story, styles, stats)
52
53     # 4. Tabla detalle por tarea
54     _add_task_table(story, styles, stats)

```

```

55
56     # 5. Tabla detalle por producto
57     _add_product_table(story, styles, stats)
58
59     doc.build(story)
60
61
62     # --- Helpers internos -----
63     -----
64
65 def _build_styles() -> dict:
66     """Retorna un diccionario con los estilos de párrafo."""
67     base = getSampleStyleSheet()
68     return {
69         "title": base["Title"],
70         "heading": base["Heading2"],
71         "body": base["Normal"],
72         "centered": ParagraphStyle(
73             "centered", parent=base["Normal"], alignment=TA_CENTER
74         ),
75     }

```

¿Por qué este diseño?

Dividir la generación en funciones privadas `_add_*` hace que el código sea fácil de depurar: si una sección da error, sabes exactamente dónde buscar.

Implementar cada sección del reporte

6.1 Encabezado y datos generales

```

1 def _add_header(story, styles, stats):
2     story.append(Paragraph("LinProd - Reporte de Simulación",
3                             styles["title"]))
4     story.append(HRFlowable(width="100%", thickness=1,
5                             color=colors.HexColor("#4f8cff")))
6
7     story.append(Spacer(1, 0.4 * cm))
8
9     resumen = [
10         ["Ciclos totales ejecutados:",
11          str(stats["total_cycles_run"])],
12         ["Productos en la línea:",
13          str(stats["products_total"])],
14         ["Productos completados:",

```

```

13         str(stats["products_completed"])),
14     ]
15     t = Table(resumen, colWidths=[9 * cm, 7 * cm])
16     t.setStyle(TableStyle([
17         ("FONT",      (0, 0), (-1, -1), "Helvetica"),
18         ("FONTSIZE",  (0, 0), (-1, -1), 11),
19         ("FONTNAME",  (0, 0), (0, -1),  "Helvetica-Bold"),
20         ("TEXTCOLOR", (0, 0), (0, -1),  colors.HexColor("#004680")
21         ),
22         ("BOTTOMPADDING", (0, 0), (-1, -1), 5),
23     ]))
24     story.append(t)
25     story.append(Spacer(1, 0.6 * cm))

```

6.2 Resumen de tiempos

```

1 def _add_summary(story, styles, stats):
2     story.append(Paragraph("Resumen de Tiempos", styles["heading"]
3     ))
4     story.append(Spacer(1, 0.2 * cm))
5
6     def fmt(v):
7         return "-" if v is None else str(v)
8
9     data = [
10         ["Métrica", "Valor"],
11         ["Primer producto completado (ciclo)",
12          fmt(stats["first_completion_cycle"])],
13         ["Último producto completado (ciclo)",
14          fmt(stats["last_completion_cycle"])],
15         ["Tiempo promedio de paso",
16          f"{stats['avg_completion_time']:.2f} ciclos"],
17         ["Tiempo total de procesamiento",
18          f"{stats['total_processing_time']} ciclos"],
19         ["Espera promedio global",
20          f"{stats['avg_wait_to_start']:.2f} ciclos"],
21     ]
22
23     t = Table(data, colWidths=[10 * cm, 6 * cm])
24     t.setStyle(TableStyle([
25         # Encabezado
26         ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#004680")
27         ),
28         ("TEXTCOLOR",    (0, 0), (-1, 0), colors.white),
29         ("FONTNAME",     (0, 0), (-1, 0), "Helvetica-Bold"),

```

```

28     ("FONTSIZE",      (0, 0), (-1, 0), 11),
29     # Cuerpo
30     ("FONTNAME",      (0, 1), (-1, -1), "Helvetica"),
31     ("FONTSIZE",      (0, 1), (-1, -1), 10),
32     ("ROWBACKGROUNDS", (0, 1), (-1, -1),
33      [colors.HexColor("#f0f4ff"), colors.white]),
34     ("GRID", (0, 0), (-1, -1), 0.5, colors.HexColor("#cccccc")
35      ),
36     ("BOTTOMPADDING", (0, 0), (-1, -1), 5),
37     ("TOPPADDING",    (0, 0), (-1, -1), 5),
38 ]))
39 story.append(t)
40 story.append(Spacer(1, 0.6 * cm))

```

6.3 Cuello de botella

```

1 def _add_bottleneck(story, styles, stats):
2     story.append(Paragraph("Cuello de Botella", styles["heading"])
3     )
4     story.append(Spacer(1, 0.2 * cm))
5
6     bneck_proc = stats.get("bottleneck_process") or "-"
7     bneck_task = stats.get("bottleneck_task") or "-"
8     worst_val = stats.get("worst_wait_value", 0.0)
9     worst_task = stats.get("worst_wait_task") or "-"
10
11     texto = (
12         f"El cuello de botella detectado es la tarea <b>{
13             bneck_task}</b> "
14         f"del proceso <b>{bneck_proc}</b>, que registró la mayor "
15         f"espera promedio en cola ({worst_val:.2f} ciclos por
16         producto)."
17     )
18     story.append(Paragraph(texto, styles["body"]))
19     story.append(Spacer(1, 0.6 * cm))

```

6.4 Tabla de detalle por tarea

```

1 def _add_task_table(story, styles, stats):
2     story.append(Paragraph("Detalle por Tarea", styles["heading"])
3     )
4     story.append(Spacer(1, 0.2 * cm))

```

```

5     encabezado = [["Proceso", "Tarea", "t (ciclos)",
6                   "Procesados", "Espera prom."]]
7
8     filas = []
9     for ts in stats["task_stats"]:
10         filas.append([
11             ts["process_name"],
12             ts["task_name"],
13             str(ts["process_time"]),
14             str(ts["products_processed"]),
15             f"{ts['avg_wait_cycles']:.2f}",
16         ])
17
18     data = encabezado + filas
19     col_w = [4*cm, 4*cm, 2.5*cm, 2.5*cm, 3*cm]
20     t = Table(data, colWidths=col_w)
21     t.setStyle(TableStyle([
22         ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#004680"),
23         ),
24         ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
25         ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
26         ("FONTNAME", (0, 1), (-1, -1), "Helvetica"),
27         ("FONTSIZE", (0, 0), (-1, -1), 9),
28         ("ROWBACKGROUNDS", (0, 1), (-1, -1),
29         [colors.HexColor("#f0f4ff"), colors.white]),
30         ("GRID", (0, 0), (-1, -1), 0.5, colors.HexColor("#cccccc")
31         ),
32         ("ALIGN", (2, 0), (-1, -1), "CENTER"),
33         ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
34         ("TOPPADDING", (0, 0), (-1, -1), 4),
35     ]))
36     story.append(t)
37     story.append(Spacer(1, 0.6 * cm))

```

6.5 Tabla de detalle por producto

```

1 def _add_product_table(story, styles, stats):
2     story.append(Paragraph("Detalle por Producto", styles["heading"]
3     ))
4     story.append(Spacer(1, 0.2 * cm))
5
6     encabezado = [["Producto", "Entrada", "Salida",
7                   "Total (ciclos)", "Estado"]]
8
9     filas = []
10    for p in stats["product_stats"]:
11        filas.append([

```



```

10         f"P{p['product_id']}",
11         str(p["entry_time"]),
12         str(p["exit_time"]) if p["exit_time"] is not None else
            "-",
13         str(p["total_time"]) if p["total_time"] is not None
            else "-",
14         "Completado" if p["is_completed"] else "En proceso",
15     ])
16
17 data = encabezado + filas
18 col_w = [2.5*cm, 2.5*cm, 2.5*cm, 3.5*cm, 5*cm]
19 t = Table(data, colWidths=col_w)
20 t.setStyle(TableStyle([
21     ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#004680"
22     )),
23     ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
24     ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
25     ("FONTNAME", (0, 1), (-1, -1), "Helvetica"),
26     ("FONTSIZE", (0, 0), (-1, -1), 9),
27     ("ROWBACKGROUNDS", (0, 1), (-1, -1),
28     [colors.HexColor("#f0f4ff"), colors.white]),
29     ("GRID", (0, 0), (-1, -1), 0.5, colors.HexColor("#cccccc"
30     )),
31     ("ALIGN", (1, 0), (-1, -1), "CENTER"),
32     ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
33     ("TOPPADDING", (0, 0), (-1, -1), 4),
34 ]))
35 story.append(t)

```

Integración con la GUI (app.py)

El botón «= **Estadísticas**» ya existe en la barra de controles. Para agregar la generación de PDF hay que añadir un botón «**PDF**» en esa misma barra y conectarlo al siguiente handler:

```
1 # En app.py, importar al inicio:
2 from tkinter import filedialog
3 from linprod.reports import generate_report
4
5 # Handler del nuevo botón:
6 def _on_generate_pdf(self) -> None:
7     if self._sim is None or not self._sim.is_started:
8         messagebox.showinfo("Sin datos",
9                             "Inicie y complete una simulación
10                             primero.")
11     return
```

```

11     stats = self._sim.get_stats()
12     path = filedialog.asksaveasfilename(
13         defaultextension=".pdf",
14         filetypes=[("PDF", "*.pdf")],
15         initialfile="reporte_linprod.pdf",
16     )
17     if not path:
18         return # usuario canceló
19     try:
20         generate_report(stats, path)
21         messagebox.showinfo("PDF generado",
22                             f"Reporte guardado en:\n{path}")
23     except Exception as e:
24         messagebox.showerror("Error al generar PDF", str(e))

```

Y en `_build_controls_card`, después del botón de estadísticas:

```

1 self._btn_pdf = ModernButton(btns, text="PDF Generar PDF",
2                               variant="secondary",
3                               command=self._on_generate_pdf)
4 self._btn_pdf.pack(side="left", padx=6)

```

Prueba del reporte sin necesidad de la GUI

Para probar `reports.py` de forma independiente, sin abrir la interfaz gráfica, puedes agregar un bloque `if __name__ == "__main__":` al final del archivo:

```

1 if __name__ == "__main__":
2     from linprod.model import Process, ProductionLine, Task
3     from linprod.simulator import Simulator
4
5     # Línea de prueba mínima
6     line = ProductionLine()
7
8     p1 = Process("Corte", is_initial=True)
9     p1.add_task(Task("Corte láser", 2))
10    p1.add_task(Task("Pulido", 2))
11
12    p2 = Process("Ensamble")
13    p2.add_task(Task("Armado", 3))
14    p2.add_task(Task("Soldadura", 4))
15
16    p3 = Process("Empaque", is_final=True)
17    p3.add_task(Task("Empacar", 2))
18    p3.add_task(Task("Etiquetado", 1))

```

```

19
20     line.add_process(p1)
21     line.add_process(p2)
22     line.add_process(p3)
23
24     sim = Simulator(line, n_products=5)
25     sim.start()
26     sim.run()      # corre todos los ciclos hasta terminar
27
28     stats = sim.get_stats()
29     generate_report(stats, "reporte_prueba.pdf")
30     print("PDF generado: reporte_prueba.pdf")

```

Para ejecutarlo desde la raíz del repositorio:

```
python3 -m linprod.reports
```

Errores comunes y cómo resolverlos

Error	Solución
ModuleNotFoundError: reportlab	Instalar con <code>pip3 install reportlab</code>
PermissionError al guardar el PDF	El archivo ya está abierto en el visor. Ciérralo antes de generarlo de nuevo.
KeyError: 'alguna_clave'	Verificar que estás accediendo a la clave correcta del diccionario (revisar §3 de esta guía).
Las tildes se ven como ? en el PDF	Usar <code>pdfmetrics.registerFont</code> con una fuente TTF que soporte UTF-8, o evitar tildes en los textos generados.
Tabla cortada al final de la pá- gina	Usar <code>KeepTogether</code> para proteger tablas cortas, o <code>TableStyle</code> con <code>splitByRow=1</code> para las largas.

Consejo sobre tildes

La fuente por defecto de reportlab (Helvetica) no soporta tildes directamente. La solución más sencilla es usar la fuente del sistema **DejaVu** que sí soporta UTF-8:

```
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont

pdfmetrics.registerFont(
    TTFont("DejaVu", "/usr/share/fonts/truetype/dejavu/
            DejaVuSans.ttf")
)
# Después usas "DejaVu" como nombre de fuente en TableStyle/
ParagraphStyle
```

Lista de verificación antes de la entrega

- ✓ `reports.py` importa únicamente de `linprod.simulator`, `reportlab` y la `stdlib`. No toca `app.py`.
- ✓ La función pública es `generate_report(stats, output_path)`.
- ✓ El PDF incluye los 7 ítems obligatorios del enunciado (§4).
- ✓ Las tablas tienen encabezado con fondo de color para legibilidad.
- ✓ Los valores `None` se muestran como “—” (no como la cadena `"None"`).
- ✓ El módulo puede ejecutarse de forma autónoma (`python3 -m linprod.reports`) para pruebas sin GUI.
- ✓ Se probó con la línea demo de 5 procesos $\times$ 2 tareas $\times$ 5 productos.
- ✗ **No** modificar `model.py`, `simulator.py` ni `app.py` (salvo agregar el botón PDF indicado en §7).